

✓ K-Nearest Neighbor Classification – Pattern Recognition Mini Project

```
# Importing Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import MinMaxScaler, StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
```

```
# Loading breast cancer dataset from sklearn
df = load_breast_cancer(as_frame=True)

# Assigning feature and target variables
feat = df.data
y = df.target
```

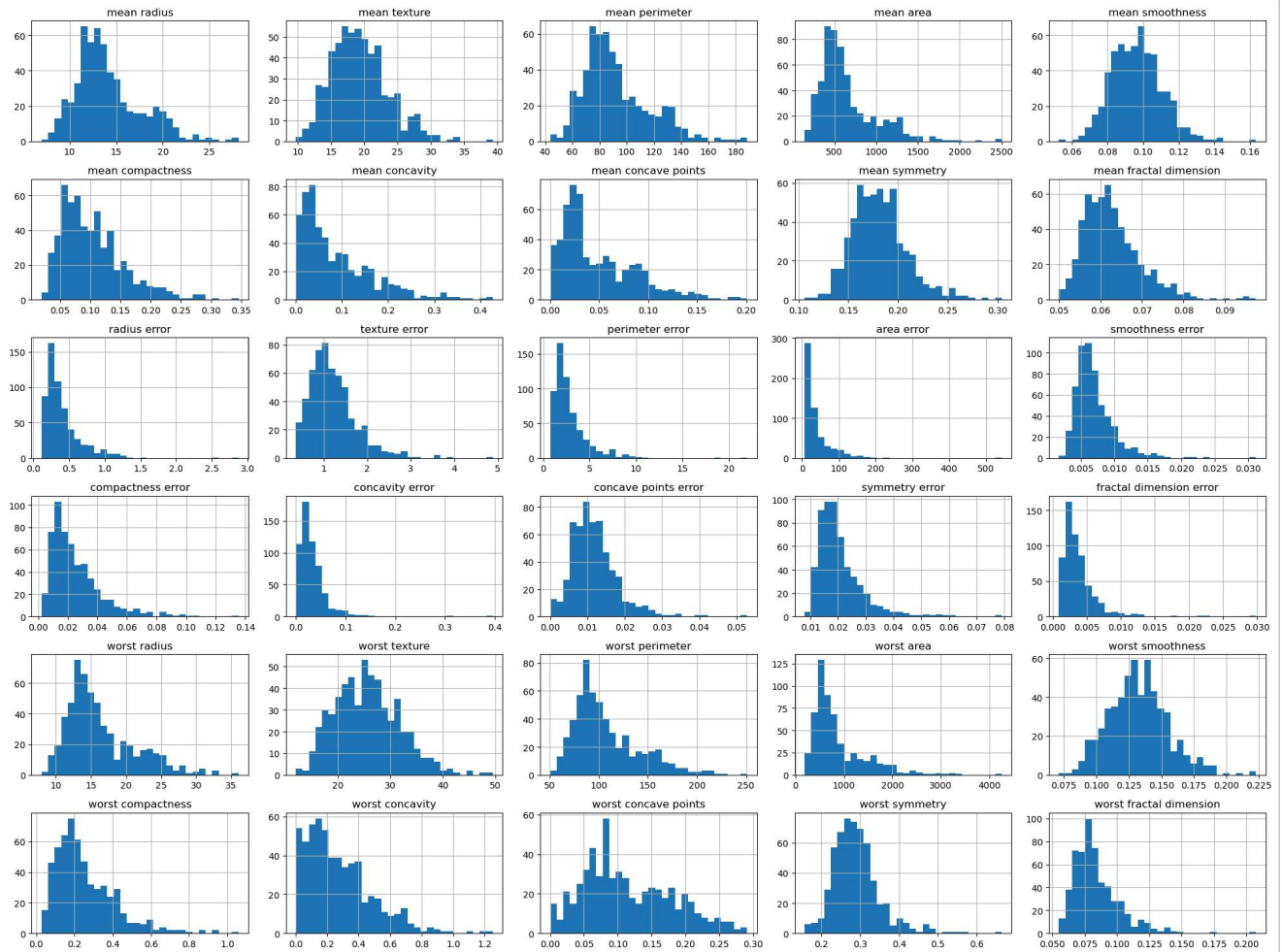
```
# Checking for missing values

print('Missing values in independent variables')
print(feat.isna().sum())
print('='*50)
print('Missing values in dependent variable')
print(y.isna().sum())
```

```
Missing values in independent variables
mean radius      0
mean texture     0
mean perimeter   0
mean area        0
mean smoothness  0
mean compactness 0
mean concavity   0
mean concave points 0
mean symmetry    0
mean fractal dimension 0
radius error     0
texture error    0
perimeter error  0
area error       0
smoothness error 0
compactness error 0
concavity error  0
concave points error 0
symmetry error   0
fractal dimension error 0
worst radius     0
worst texture    0
worst perimeter  0
worst area       0
worst smoothness 0
worst compactness 0
worst concavity  0
worst concave points 0
worst symmetry   0
worst fractal dimension 0
dtype: int64
=====
Missing values in dependent variable
0
```

```
# Visualizing feature distribution

feat.hist(bins=30,figsize=(20, 15))
plt.tight_layout()
plt.show()
```



▼ Data Preprocessing - Feature Scaling

```
# Normalizing features
min_max = MinMaxScaler()
x = min_max.fit_transform(feats)
```

▼ Splitting data

```
# Splitting data into 80% train and 20% test
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.2,random_state=42)
```

▼ Model training

```
# Training KNN Model with k value of 6
knn = KNeighborsClassifier(n_neighbors=6)
knn.fit(x_train,y_train)
```

▼ KNeighborsClassifier ⓘ ?

► Parameters

▼ Model Evaluation

```
# Predicting of x_test data subset
y_pred = knn.predict(x_test)

accuracy = accuracy_score(y_test,y_pred)
precision = precision_score(y_test,y_pred)
recall = recall_score(y_test,y_pred)
f1 = f1_score(y_test,y_pred)
cm = confusion_matrix(y_test,y_pred)

print('Model Performance Metrics')
print('='*35)
print(f"Accuracy: {accuracy:.2f}")
print('='*35)
print(f"Precision: {precision:.2f}")
print('='*35)
print(f"Recall: {recall:.2f}")
print('='*35)
print(f"F1 Score: {f1:.2f}")
print('='*35)
print('Confusion Matrix')
print('='*35)
print(cm)
```

```
Model Performance Metrics
=====
Accuracy: 0.96
=====
Precision: 0.97
=====
Recall: 0.96
=====
F1 Score: 0.96
=====
Confusion Matrix
=====
[[41  2]
 [ 3 68]]
```

✓ Cross-Validation & Hyperparameter tuning

```
# Parameters for grid search
param_grid = {
    'n_neighbors': [1, 3, 5, 7, 9, 11]
}

# Initializing grid search cross validation
grid_search = GridSearchCV(
    estimator=KNeighborsClassifier(),
    param_grid=param_grid,
    scoring='accuracy'
)

grid_search.fit(x_train,y_train)

k_values = grid_search.cv_results_['param_n_neighbors'].data
mean_accuracy = grid_search.cv_results_['mean_test_score']

plt.figure(figsize=(8,5))

plt.plot(k_values, mean_accuracy, marker='o')

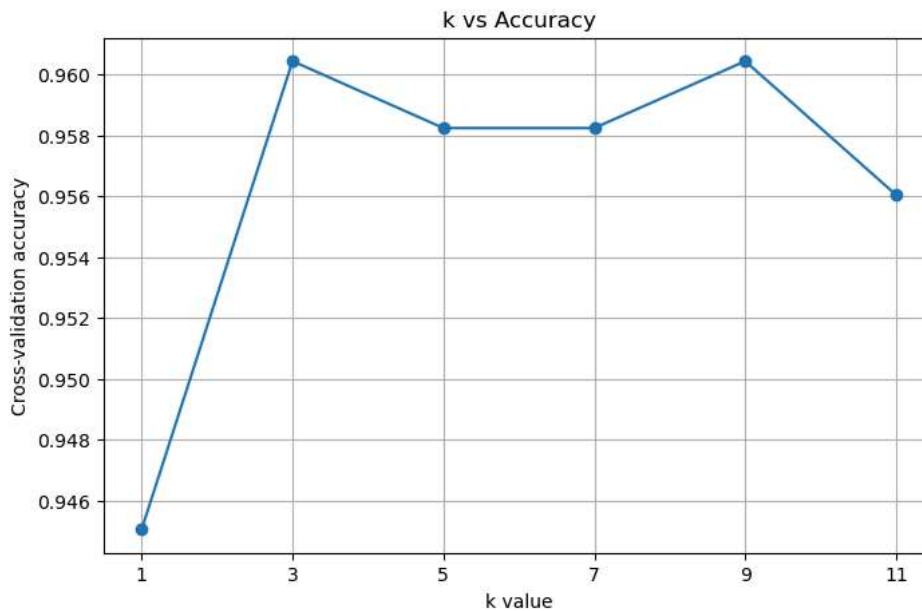
plt.xlabel('k value')
plt.ylabel('Cross-validation accuracy')
plt.title('k vs Accuracy')

plt.xticks(k_values)

plt.grid(True)

plt.show()

print('Best k:', grid_search.best_params_)
```



Best k: {'n_neighbors': 3}

Based on the result of the grid search cross validation, the model will perform better when k value is 3

Final Model

```
k3_knn = KNeighborsClassifier(n_neighbors=3)
final_model = k3_knn.fit(x_train,y_train)

y_pred_k3 = k3_knn.predict(x_test)

k3_accuracy = accuracy_score(y_test,y_pred_k3)
k3_precision = precision_score(y_test,y_pred_k3)
k3_recall = recall_score(y_test,y_pred_k3)
k3_f1 = f1_score(y_test,y_pred_k3)
k3_cm = confusion_matrix(y_test,y_pred_k3)
```

```
print('Final Model Performance Metrics')
print('='*35)
print(f"Accuracy: {k3_accuracy:.2f}")
print('='*35)
print(f"Precision: {k3_precision:.2f}")
print('='*35)
print(f"Recall: {k3_recall:.2f}")
print('='*35)
print(f"F1 Score: {k3_f1:.2f}")
print('='*35)
print('Final Model Confusion Matrix')
print('='*35)
print(k3_cm)
```

```
Final Model Performance Metrics
=====
Accuracy: 0.96
=====
Precision: 0.97
=====
Recall: 0.97
=====
F1 Score: 0.97
=====
Final Model Confusion Matrix
=====
[[41  2]
 [ 2 69]]
```

Testing prediction on new sample data

```
np.random.seed(42)

new_data = np.random.uniform(0.02, 0.40, 30)

prediction = k3_knn.predict(new_data.reshape(1, -1))
print(prediction)
```

```
[1]
```

The model predicted the outcome of the new dataset is a benign breast cancer case

✓ **Impact of Feature Scaling and Distance Metrics on KNN Behavior**

KNN is a distance-based algorithm, so both feature scaling and the choice of distance metric greatly affect its performance.

Feature scaling is important because features with larger numerical values can dominate the distance calculations, causing smaller-scale features to have little influence. Techniques such as Standardization or Normalization ensure that all features contribute equally.

Distance metrics determine how similarity between data points is measured. Common metrics include Euclidean distance (straight-line distance), Manhattan distance (sum of absolute differences), and Minkowski distance (a generalization of both). Different metrics can produce different nearest neighbors and affect the model's accuracy.

In summary, proper feature scaling and an appropriate distance metric improve the accuracy and reliability of KNN predictions.